Smart office preferences manager

PRIOT Project

Author:

Dragomir Andrei-Mihai 343C1

Description:

Documentation for the design and implementation of a smart in office logger, an IoT system for managing employee access and monitoring environmental conditions. The system integrates hardware components such as RFID, sensors, and an ESP32 microcontroller, and communicates with a server for centralized data visualization and management.

December 29, 2024

Contents

Chapter 1

Introduction

This documentation describes the implementation of an IoT system for managing employee access and monitoring the environment (temperature and humidity). The project combines a sensor network with an ESP32 hub that communicates with a central server to store and visualize data.

Employees receive alerts when the temperature or humidity is not optimal for their inputted preferences. The system also logs access events, allowing administrators to monitor employee activity: logins, logouts, access attempts, time spent in the office, etc.

1.1 Key Objectives

- Manage employee access through an RFID system.
- Monitor the environment using a DHT11 sensor.
- Visualize collected data via an interactive web interface.

Chapter 2

Architecture

2.1 Overview

This project architecture is designed to achieve modularity and scalability by separating the ESP32 microcontroller logic from the backend server and frontend interface. It uses a combination of hardware, communication protocols, and software layers to deliver the functionality.

2.1.1 Backend (Flask Server)

The backend is implemented using Python and Flask. It serves as the central hub for processing data received from the ESP32, managing user preferences, and rendering the web-based interface for administrators and employees. Key components include:

- **server.py**: The Flask application that handles HTTP requests and serves dynamic HTML pages.
- **templates/**: Contains HTML files used by Flask for dynamic content rendering (e.g., `preferences.html`, `records.html`).
- **static/**: Stores frontend assets such as:
  - **CSS**: Styling for the interface (e.g., `style.css`).
  - **JavaScript**: Frontend logic and interactivity (e.g., `script.js`).
- **users.json**: A lightweight storage file for user data and preferences.

2.1.2 ESP32 Firmware (C++)

The ESP32 firmware, written in C++, handles the hardware peripherals and communicates with the Flask server. It is organized into several files within the `src/` directory:

- **main.cpp**: The entry point for the ESP32 firmware, initializing all components and managing the main logic loop.
- **access_utils.cpp**: Handles RFID-based access control and logs employee activities such as logins and logouts.
- **http_utils.cpp**: Manages HTTP communication between the ESP32 and the Flask server for data exchange.
- **led_buzz_utils.cpp**: Controls visual and auditory feedback through RGB LEDs and a buzzer.
- **temp_utils.cpp**: Captures and processes temperature and humidity data using the DHT11 sensor.
- **rtc_utils.cpp**: Provides accurate timestamps using the RTC module.
- **print_utils.cpp** and **utils.hpp**: Utility functions for logging, debugging, and reusability.

2.2 Hardware and Network Schema

The system uses an ESP32 microcontroller to interact with peripherals and communicate with the backend server over Wi-Fi. The topology is represented as follows:

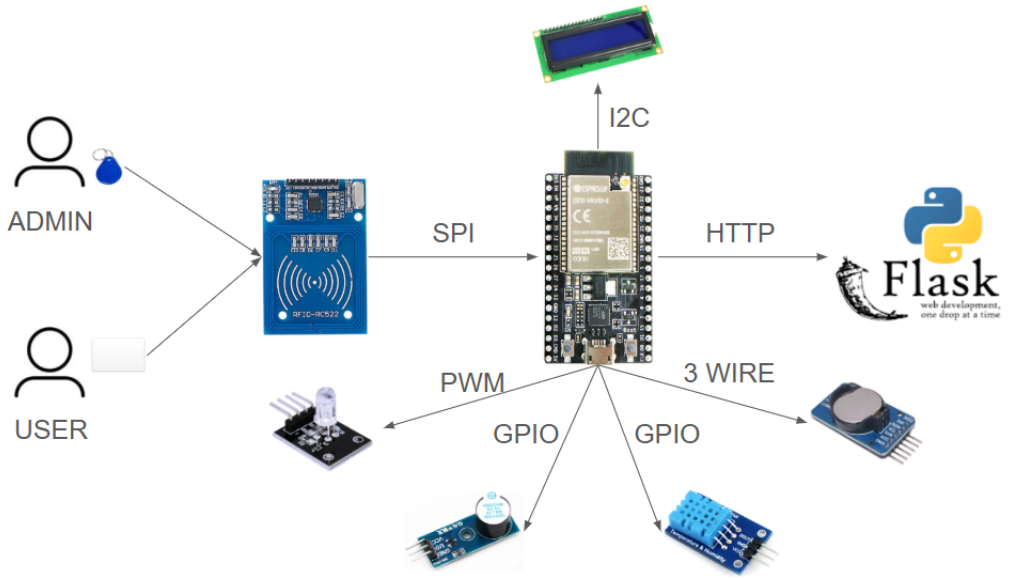

Figure 2.1: Hardware Schema

2.2.1 Hardware and Protocols

The PRIOT system is designed to integrate various hardware components with the ESP32 microcontroller, which serves as the central hub for communication and control. The following describes the key components and their connections:

- **RFID Reader (SPI Protocol):** The RFID module communicates with the ESP32 using the high-speed SPI protocol. It is used for user authentication and access control. Admins and users interact with the system by scanning their RFID tags or cards.
- **LCD Screen (I2C Protocol):** The LCD display is connected to the ESP32 via the I2C protocol. It provides a user interface for displaying system status, access logs, and environmental data locally.
- **Flask Server (HTTP over Wi-Fi):** The ESP32 communicates with the Flask server using the HTTP protocol over Wi-Fi. This connection

is used to send collected data (e.g., RFID access logs and environmental readings) and receive configurations or updates from the server.

- **RGB LED (PWM):** The RGB LED is connected to the ESP32 through GPIO pins configured for PWM (Pulse Width Modulation). It provides visual feedback for operations such as successful logins, errors, or status updates.
- **Buzzer (GPIO):** The buzzer is directly connected to a GPIO pin on the ESP32. It provides auditory feedback for system events, such as login success, login failure, or alerts.
- **DHT11 Sensor (GPIO):** The DHT11 sensor is linked to the ESP32 through a GPIO pin. It captures environmental data, specifically temperature and humidity, which is logged during user interactions.
- **RTC Module (Three-Wire Protocol):** The Real-Time Clock (RTC) module connects to the ESP32 using a three-wire protocol, providing accurate timestamps for logging access events and environmental data.

This hardware configuration ensures good communication and reliable data exchange between the various components, with the ESP32 acting as the central coordinator.

Chapter 3

Implementation

This system is implemented using a combination of C++ for the ESP32 firmware and Python for the Flask server. The following sections detail the key components and their functionalities.

3.1 ESP32 Firmware

3.2 Server Backend

3.3 API interactions

Chapter 4

Visualization and Data Processing

4.1 Web Interface

4.2 Data Processing

4.3 User Preferences

4.4 Graphics

4.5 Alerts

Chapter 5

Security

5.1 Access Control

5.2 Protocols

5.3 Data Encryption